coulomb_function

February 9, 2026

0.1 Analysis of the Coulomb Failure Function

Here, I extract fluid pressure, normal stress, and shear stress along the fracture as functions of space and time, and use them to compute the Coulomb Failure Function (CFF).

No failure criterion is enforced in the 3DEC model; the CFF is therefore used purely as a diagnostic indicator of proximity to failure.

0.1.1 Coulomb Failure Function

The Coulomb Failure Function is defined as:

$$\mathrm{CFF}(x,t) = \tau(x,t) - \mu\left(\sigma_n(x,t) - P(x,t)\right),$$

Interpretation: - **CFF < 0**: stable conditions
- **CFF $= 0$**: onset of failure
- **CFF > 0**: failure condition

0.1.2 Contribution of each variable to the CFF

To assess which physical mechanism dominates the approach to failure, I compute dimensionless contribution factors for shear stress, fluid pressure, and normal stress.

The shear stress contribution is defined as:

$$\mathrm{SSC} = \frac{\tau(x,t_i)}{\mathrm{CFF}(x,t_i)}.$$

If SSC is close to 1, failure is primarily driven by shear stress.

Similarly, the fluid pressure and normal stress contributions are:

$$\mathrm{FPC} = \frac{\mu P(x,t_i)}{\mathrm{CFF}(x,t_i)}, \quad \mathrm{NSC} = \frac{\mu\sigma_n(x,t_i)}{\mathrm{CFF}(x,t_i)}.$$

Property	Value
Fracture length, L	100 m
Normal stiffness, k_n	50 GPa/m
Initial aperture, w_i	0.1 mm
Fluid viscosity, μ_w	10^{-3} Pa · s
Duration, T	100 s
Friction, μ	0.6
Young's modulus, E	60 GPa
Poisson's ratio, ν	0.25

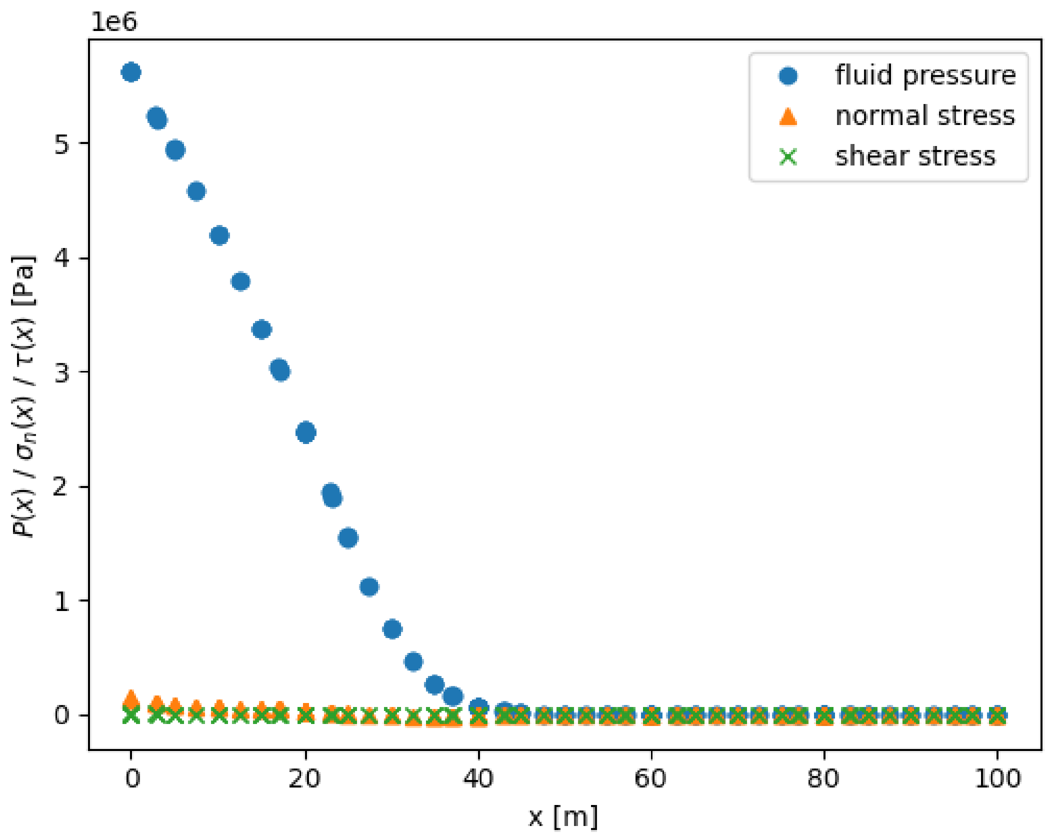
[12]:
```
from pathlib import Path
from typing import cast

import h5py
import matplotlib.pyplot as plt
import numpy as np
```

[13]:
```
# ----------------------------------------------------------------------------
# Load 3DEC output data
# ----------------------------------------------------------------------------

filepath = Path.cwd() / "results" / "3dec" / "runs" / "run_01.hdf5"

with h5py.File(filepath, "r") as f:
    x_subcontacts = cast(h5py.Dataset, f["coordinates/x_subcontacts"])[:]
    p_tx = cast(h5py.Dataset, f["fields/fluid_pressure"])[:]
    sn_tx = cast(h5py.Dataset, f["fields/stress_normal"])[:]
    tau_tx = cast(h5py.Dataset, f["fields/stress_shear"])[:]
    t_points = cast(h5py.Dataset, f["coordinates/t"])[:]
```

[14]:
```
# ----------------------------------------------------------------------------
# Select time for spatial analysis
# ----------------------------------------------------------------------------

t = 20.0  # target time [s]
idx_t = np.searchsorted(t_points, t)

p_x = p_tx[idx_t]
sn_x = sn_tx[idx_t]
tau_x = tau_tx[idx_t]
```

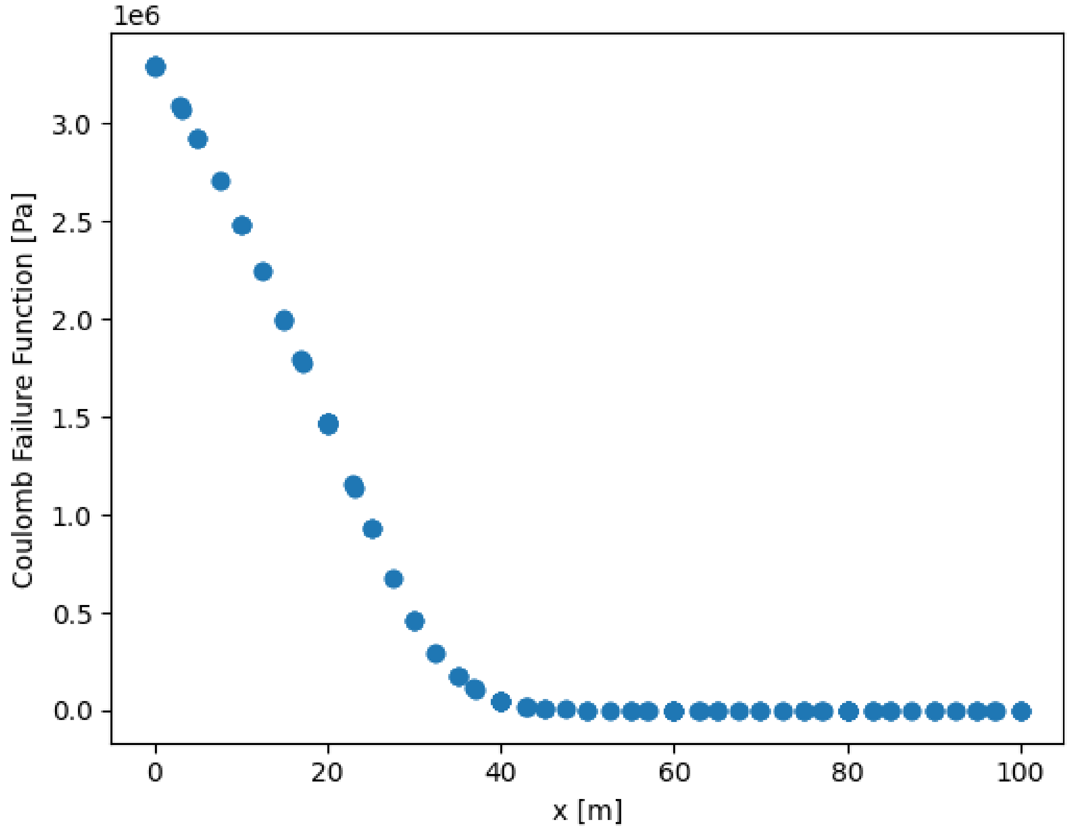
[15]:
```
# ----------------------------------------------------------------------------
# Plot pressure and stresses along the fracture
# ----------------------------------------------------------------------------

fig, ax = plt.subplots()
```

```python
ax.plot(x_subcontacts, p_x, "o", label="fluid pressure")
ax.plot(x_subcontacts, sn_x, "^", label="normal stress")
ax.plot(x_subcontacts, tau_x, "x", label="shear stress")

ax.set_xlabel("x [m]")
ax.set_ylabel(r"$P(x)$ / $\sigma_n(x)$ / $\tau(x)$ [Pa]")
ax.legend()
plt.show()
```



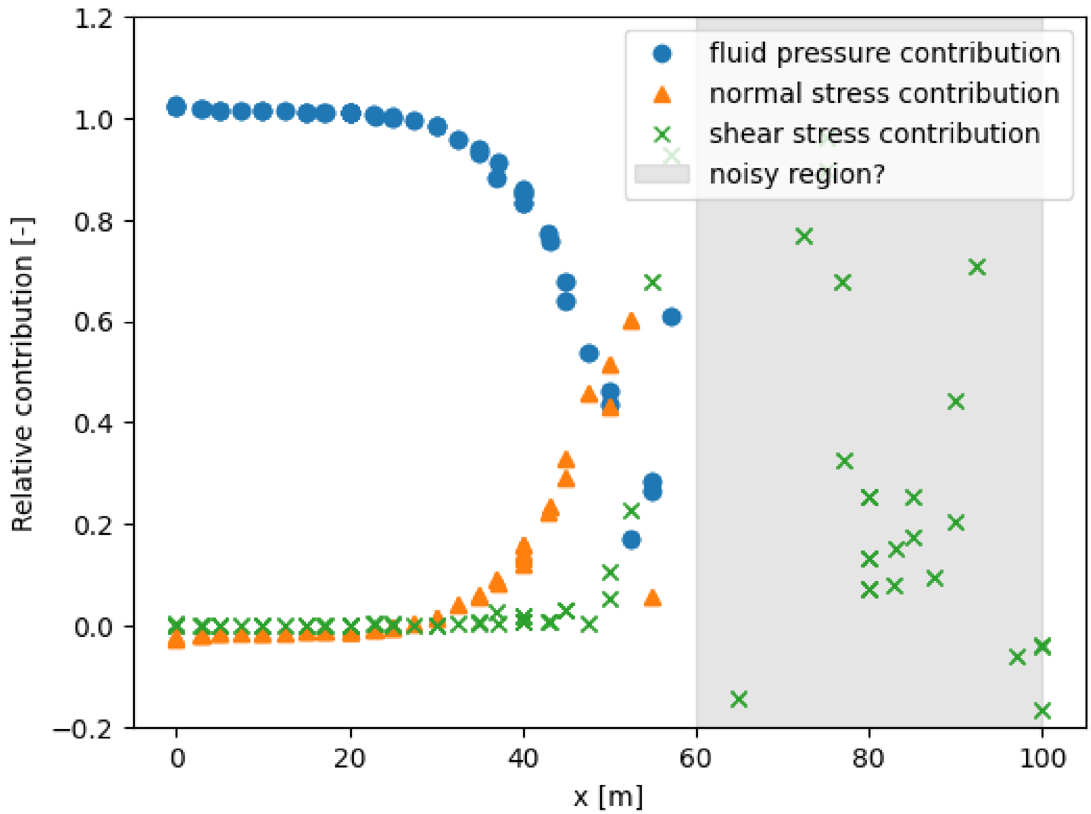
[16]:
```python
# ---------------------------------------------------------------------------
# Compute and plot Coulomb Failure Function
# ---------------------------------------------------------------------------

mu = 0.6

CFF = tau_x - mu * (sn_x - p_x)

fig, ax = plt.subplots()
ax.plot(x_subcontacts, CFF, "o")
```

```
ax.set_xlabel("x [m]")
ax.set_ylabel("Coulomb Failure Function [Pa]")
plt.show()
```



[17]:
```
# --------------------------------------------------------------------------
# Contribution analysis
# --------------------------------------------------------------------------

SSC = tau_x / CFF
FPC = mu * p_x / CFF
NSC = - mu * sn_x / CFF

fig, ax = plt.subplots()
ax.plot(x_subcontacts, FPC, "o", label="fluid pressure contribution")
ax.plot(x_subcontacts, NSC, "^", label="normal stress contribution")
ax.plot(x_subcontacts, SSC, "x", label="shear stress contribution")
ax.axvspan(
    60,
    x_subcontacts.max(),
    color="gray",
```

```
    alpha=0.2,
    label="noisy region?"
)
ax.set_ylim((-0.2, 1.2))
ax.set_xlabel("x [m]")
ax.set_ylabel("Relative contribution [-]")
ax.legend(loc='upper right')
plt.show()
```



[ ]: